

DEPARTMENT OF COMPUTER SCIENCE

Web Engineering (5th Semester)

LAB 04 SUBMISSION

CSS Layout: Flexbox, CSS Grid, Mobile-First Breakpoints & Positioning

Student Name:	Muhammad Tayyab
CMS ID:	023-24-0118
Course:	Web Engineering
Semester:	5th Semester — Fall 2026
Lab Deliverable:	Week 04: CSS Layout (Flexbox & Grid)
GitHub Repository:	github.com/taybro-o/web-engineering-lab

Note: All hands-on tasks, responsive layout drills, and positioning tests have been implemented, tested locally in Google Chrome DevTools, and committed to the GitHub repository under Week 04/.

1. Warm-up: Flexbox (1D) vs Grid (2D)

Goal & Execution: Opened warmup.html to verify how Flexbox and Grid behave with the same set of elements. I added a seventh box (G) to both sections and tested flex-direction: column on the flex container.

- **Task 1 (Add box G):** In Flexbox (display: flex), all 7 boxes (A–G) remained in a single horizontal row, shrinking in width to fit. In CSS Grid (grid-template-columns: 1fr 1fr 1fr), box G automatically wrapped and started a new 3rd row right under box A.
- **Task 2 (flex-direction: column):** Adding flex-direction: column turned the flex section into a single vertical column. The Grid section remained unaffected.

Core Rule: *Flexbox is 1-dimensional (controls either a row OR a column). Grid is 2-dimensional (controls rows AND columns simultaneously).*

```
[Week 04/warmup.html]

<!-- warmup.html (excerpt) -->
<style>
  .flex-row { display: flex; gap: 0.5rem; }
  .flex-col { display: flex; flex-direction: column; gap: 0.5rem; }
  .grid    { display: grid; grid-template-columns: 1fr 1fr 1fr; gap: 0.5rem; }
</style>
<section class="flex-row">
  <div class="box">A</div> ... <div class="box">G</div>
</section>
<section class="grid">
  <div class="box">A</div> ... <div class="box">G</div> <!-- G wraps to 3rd row -->
</section>
```

2. Class Activity A: Flexbox Gallery (`gallery.html`)

Goal & Implementation: Built a responsive photo gallery using Flexbox with 6 card boxes. Applied display: flex and flex-wrap: wrap with gap: 1rem on the .gallery container, and set flex: 1 1 160px on .photo items.

Observation: The items prefer an initial base width of 160px (flex-basis), but can grow (flex-grow: 1) to fill available space or shrink (flex-shrink: 1) before wrapping. Resizing the browser window down from desktop to mobile causes the layout to gracefully reflow from 4 → 2 → 1 across, with 0 horizontal scrollbar at 320px.

```
[Activity A: Flexbox wrapping CSS]

/* gallery.html - Activity A */
.gallery {
  display: flex;
  flex-wrap: wrap;
  gap: 1rem; /* Container gap preferred over child margins */
}
.photo {
  flex: 1 1 160px; /* grow: 1, shrink: 1, basis: 160px */
  background: #ddd;
  padding: 2rem 1rem;
  text-align: center;
}
```

3. Class Activity B: justify-content Space Distribution

Goal & Testing: Set .photo { flex: 0 0 120px; } so items have a fixed width and cannot grow to absorb remaining space, allowing clear observation of justify-content on the main axis.

Value	Main Axis Behavior & Visual Observation
flex-start	Items pack tightly at the start (left) of the container; all unused space stays at the far right.
center	Items cluster together in the center of the line; leftover space is evenly split on left and right edges.
flex-end	Items pack tightly at the end (right); all leftover space sits at the beginning.
space-between	First item is flush left, last item is flush right; leftover space is distributed equally between items.
space-around	Items have equal space on both sides (space between items is twice the space at the edges).
space-evenly	All spaces between items and between items and edges are strictly equal.

4. Class Activity C: Responsive CSS Grid (`grid-gallery.html`)

Goal & Implementation: Built a responsive 6-item photo gallery using CSS Grid with auto-fit and minmax. The CSS rule used was: `display: grid; grid-template-columns: repeat(auto-fit, minmax(250px, 1fr)); gap: 1rem;`

Observation: No media queries were needed. The browser automatically calculated how many columns of at least 250px could fit across the container width and expanded them with 1fr. At viewport widths from 1200px down to 320px, columns reflowed from 4 → 3 → 2 → 1 without overflow or scrollbars.

[Activity C: CSS Grid auto-fit]

```
/* grid-gallery.html - Activity C */
.gallery {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
  gap: 1rem;
}
.photo {
  background: #ddd;
  padding: 2.5rem 1rem;
  text-align: center;
}
```

5. Class Activity D: Mobile-First Breakpoints (`breakpoints.html`)

Goal & Implementation: Replaced auto-fit with explicit mobile-first min-width media queries. The mobile base style starts at 1 column with no query, and progressively expands column count as screen width increases.

Breakpoint	Media Query	Columns	Observed Layout
Base (Mobile)	None (Base CSS)	1fr	1 column stacked layout for phones (< 640px)
Small	@media (min-width: 640px)	2 cols	2 columns side by side for large phones/small tablets
Medium	@media (min-width: 768px)	3 cols	3 columns for tablets in portrait mode
Large	@media (min-width: 1024px)	4 cols	4 columns for laptops and desktops
Extra Large	@media (min-width: 1280px)	4 cols (gap: 1.5rem)	4 columns with widened 1.5rem spacing on wide screens

6. Class Activity E: CSS Positioning (`position.html`)

Tested Scenarios:

- **Static:** Normal flow. All boxes render in HTML document order with standard gap spacing. No offsets apply.
- **Relative:** The middle navy box is shifted by top: 22px; left: 22px;. Its original space is preserved (indicated by the white dotted ghost), and neighbouring boxes do NOT collapse into the empty space.
- **Absolute:** The mint box is completely taken out of document flow. Neighbours immediately slide together to close the gap. The box is measured and positioned relative to the parent with position: relative (the red origin dot).
- **Fixed vs Sticky:** The bottom-right mint square is position: fixed; it stays anchored to the browser window throughout scrolling. The mint square inside the pink container is position: sticky; top: 8px;; it sticks to the top as long as its parent is in the viewport, and leaves the screen when the pink parent scrolls out of view.

Required Lab Task Sentence Completions:

1. *Static is the default document flow where elements are positioned in normal source order without offsets.*
2. *Relative keeps a gap; absolute leaves normal flow and lets neighbours close the gap.*
3. *Fixed follows the window; sticky follows its parent scrolling container.*

[Activity E: CSS Position Rules]

```
/* position.html - positioning rules */
.nudge { position: relative; top: 22px; left: 22px; }
.abs-frame { position: relative; min-height: 170px; }
.abs-child { position: absolute; top: 28px; left: 28px; }
.fixed-sq { position: fixed; right: 1rem; bottom: 1rem; z-index: 40; }
.sticky-sq { position: sticky; top: 8px; }
```